Lab 5: Static Code Analysis Report

Name:BR Manasi

Section:B

Srn:PES2UG24CS104

Date:14-10-2026

Tournament Scoreboard

GitHub repository: https://github.com/manasi132/L5_38_tournament_scoreboard

Branch: main | Commit: 4500179

1. Objective

The objective of this lab is to improve the tournament scoreboard source code and verify it using Pylint, Flake8, and Bandit.

2. Verification Summary

Check	Observed result
Program execution	Successful; scoreboard output displayed.
Pylint	10.00/10.
Flake8	No output; no violations reported.
Bandit	Zero reported findings.
GitHub	Commit 4500179 pushed to the main branch.

3. Screenshot Evidence

1. Program Execution

The scoreboard runs successfully and displays the team standings, points, top-two status, and match count.

```
(venv) PS C:\Users\B R MANASI\L5_38_tournament_scoreboard> python tournament_scoreboard.py
Toss: Heads
Pos Team Played Won Drawn Lost Pts
1 Royal Strikers 2 1 1 0 4
2 City Rovers 2 1 0 1 3
3 Hill Warriors 2 0 2 0 2
4 Lake United 2 0 1 1 1
Is City Rovers in the top two? True
Matches played: 4
```

2. Pylint Results

Pylint reports a score of 10.00/10.

```
branch 'main' set up to track 'myfork/main'.              python -m pylint tournament_scorebo
ard.py PS C:\Users\B R MANASI\L5_38_tournament_scoreboard>

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

(venv) PS C:\Users\B R MANASI\L5_38_tournament_scoreboard>
```

3. Flake8 Results

Flake8 produces no output, indicating no reported violations.

```
(venv) PS C:\Users\B R MANASI\L5_38_tournament_scoreboard> python -m flake8 tournament_scorebo
ard.py
(venv) PS C:\Users\B R MANASI\L5_38_tournament_scoreboard>
```

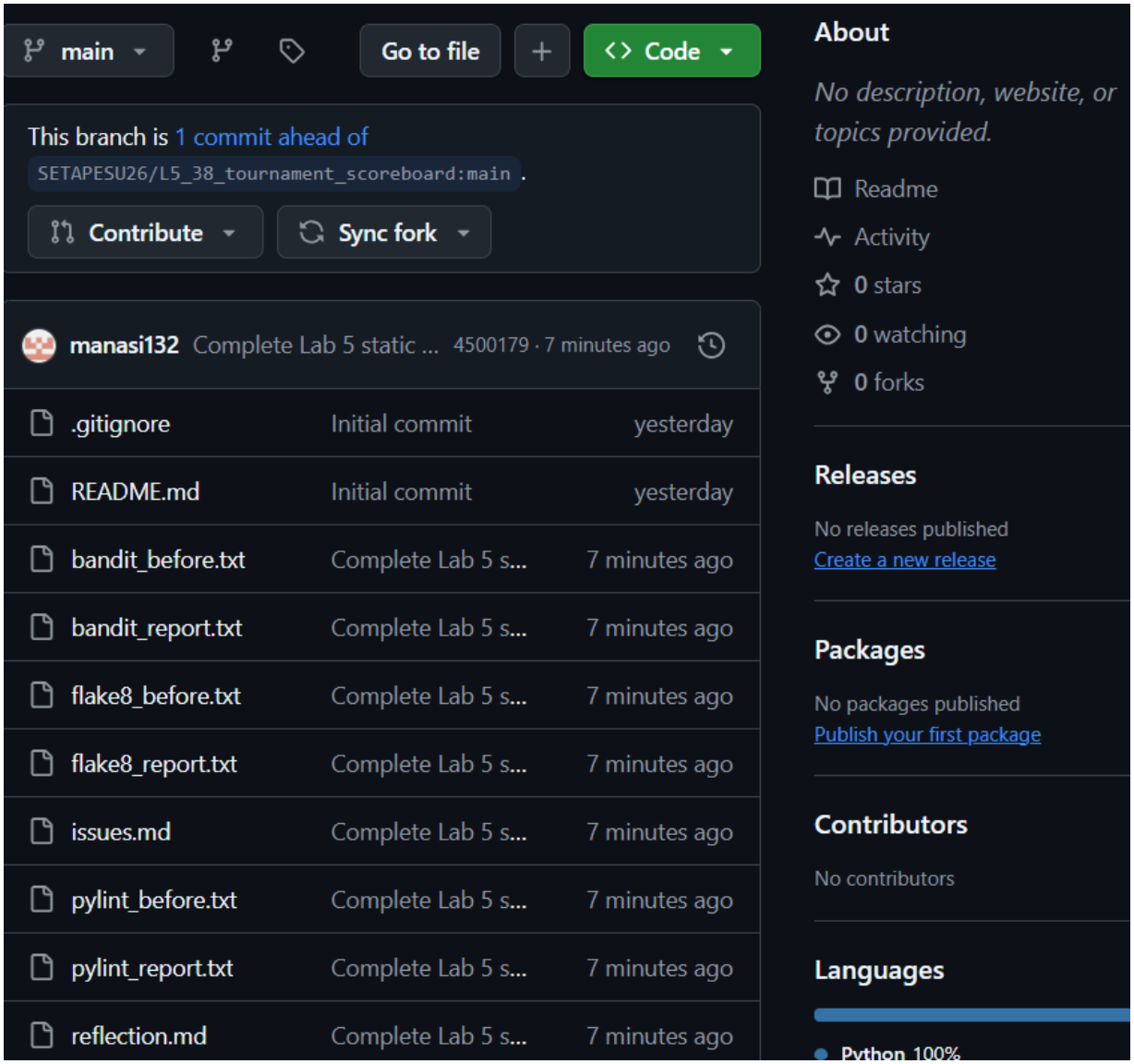
4. Git Push Confirmation

The main branch was pushed to the personal GitHub repository and set to track myfork/main.

```
(venv) PS C:\Users\B R MANASI\L5_38_tournament_scoreboard> git push -u myfork main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (12/12), 8.80 KiB | 900.00 KiB/s, done.
Total 12 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/manasi132/L5_38_tournament_scoreboard.git
   c9ef096..4500179  main -> main
branch 'main' set up to track 'myfork/main'.              python -m pylint tournament_scoreb
```

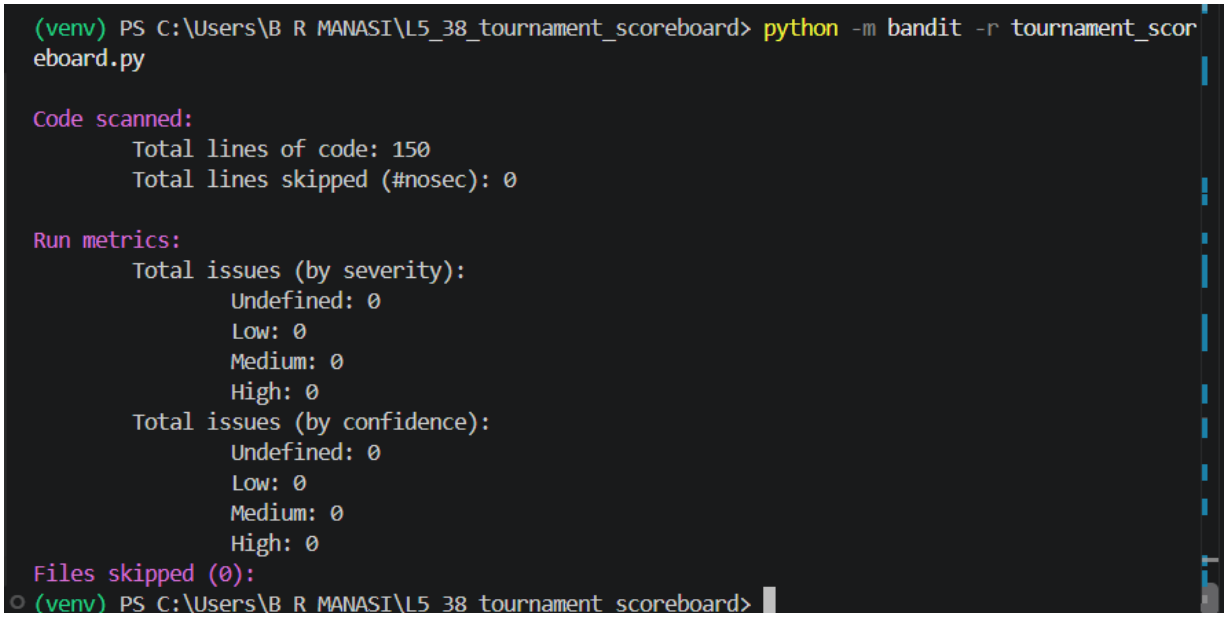
5. GitHub Repository

The repository page shows commit 4500179 and the project/report files.



6. Bandit Results

Bandit reports zero findings in the displayed severity and confidence categories.



4. Conclusion

The scoreboard executes successfully. The captured verification evidence shows a Pylint score of 10.00/10, no Flake8 output, and zero Bandit findings. The changes were pushed to the personal GitHub repository. The source code and supporting documentation are available in the repository.

5. Reflection Questions and Answers

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest issues were style and formatting problems, such as removing an unused import and adding the missing final newline. These changes were straightforward because they did not alter the program's logic. The harder issues were improving validation and making the remote publishing operation safer. Input validation required checking team names, player names, and goal values, while the publishing function needed to avoid shell execution and validate the destination before calling scp.

2. Did the static analysis tools report any false positives? If so, describe one example.

Bandit initially reported low-severity warnings B404 and B603 for importing subprocess and calling subprocess.run. These warnings are general cautions about subprocess use; they do not by themselves prove that the code is exploitable. The implementation uses an argument list and shell=False, which avoids invoking a shell. Therefore, these findings were treated as warnings requiring review rather than proof of a vulnerability. After the final code change removed the publishing code and its related imports, Bandit reported zero findings.

3. How would you integrate static analysis tools into your actual software development workflow?

I would run Flake8 and Pylint locally while developing, before committing changes, and run Bandit to check for common security concerns. I would also add these checks to a GitHub Actions workflow so they run automatically on each pull request. A change would be reviewed before merging if the checks failed, and I would keep the reports or CI results as evidence.

4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

The code became more consistent and easier to read through clearer function names, docstrings, and cleaner formatting. Validation now rejects empty team names, invalid player names, negative or non-integer goal counts, unknown teams, and a team playing against itself. Player lists are copied instead of relying on a shared mutable default. The scoreboard updates match statistics consistently, uses secrets.choice for the coin toss, and avoids shell-based command execution in the original publishing implementation. The final checks showed Pylint at 10.00/10, no Flake8 violations, and zero Bandit findings.